

Ranjit Panigrahi

AN NLP BASED EMAIL SPAM DETECTION

-  Quick Submit
-  Quick Submit
-  Sikkim Manipal University

Document Details

Submission ID**trn:oid::1:3246456735****Submission Date****May 11, 2025, 6:12 PM GMT+5:30****Download Date****May 11, 2025, 6:15 PM GMT+5:30****File Name****Email_Spam_Project_Report_1.docx****File Size****1.4 MB****24 Pages****4,845 Words****28,720 Characters**

6% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Small Matches (less than 13 words)

Exclusions

- 1 Excluded Source

Match Groups

-  **10 Not Cited or Quoted 3%**
Matches with neither in-text citation nor quotation marks
-  **1 Missing Quotations 0%**
Matches that are still very similar to source material
-  **5 Missing Citation 2%**
Matches that have quotation marks, but no in-text citation
-  **2 Cited and Quoted 1%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 4%  Internet sources
- 3%  Publications
- 4%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 10 Not Cited or Quoted 3%**
Matches with neither in-text citation nor quotation marks
- 1 Missing Quotations 0%**
Matches that are still very similar to source material
- 5 Missing Citation 2%**
Matches that have quotation marks, but no in-text citation
- 2 Cited and Quoted 1%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 4% Internet sources
- 3% Publications
- 4% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Student papers		
	Sikkim Manipal University		1%
2	Internet		
	medium.com		<1%
3	Internet		
	doaj.org		<1%
4	Publication		
	"Current and Future Trends on AI Applications", Springer Science and Business M...		<1%
5	Student papers		
	Leeds Beckett University		<1%
6	Student papers		
	University of Bradford		<1%
7	Student papers		
	University of Lancaster		<1%
8	Student papers		
	University of West London		<1%
9	Internet		
	dokumen.pub		<1%
10	Student papers		
	University of Hertfordshire		<1%

11	Internet	journal.stiemb.ac.id	<1%
12	Internet	munin.uit.no	<1%
13	Student papers	Manchester Metropolitan University	<1%
14	Publication	Yakub Kayode Saheed, Joshua Ebere Chukwuere. "XAIEnsembleTL-IoV: A new eXp...	<1%

Project Report on

AN NLP BASED EMAIL SPAM DETECTION



By

Saiyu Rajbhandari (202216081)

Christina Biswakarma (202216044)

Ayush Chettri (202216032)

In partial fulfillment of requirements for the award of degree in
Bachelor of Computer Application
(2022-2025)



Under the Project Guidance of

Dr. Ranjit Panigrahi

Asst. Prof. (SG), Coordinator R&D

Department of Computer Applications

Sikkim Manipal institute of Technology

Majhitar, Rangpo, East Sikkim

1. INTRODUCTION

Email is a universally used tool for businesses, offering universal accessibility, affordability, and reliable communication. Its widespread use and ongoing security enhancements have made it indispensable for both professional and personal interactions, though its popularity has also attracted challenges like spam and misuse [1]. Email spam or junk email, refers to unwanted messages sent in large quantities, often with malicious, deceptive, or promotional intent [2]. These emails are harmful as they are designed to gather sensitive information, exploit trust, spread false information leading inboxes to clutter hiding important messages.

In 2023, phishing scams (including spam emails) accounted for 36% of data breaches globally. The Federal Trade Commission (FTC) reported that Americans lost over \$2.7 billion in phishing and spam-related fraud in 2022 [3] Such volumes hide the severity of the problem and the need for effective communication.

Dodging phishing emails and suspicious links helps avoid accidental clicks on dangerous links. Regular inbox clean-ups help reduce clutter by unsubscribing from unwanted sender and email lists. It helps individual preserve productivity and motivation and improve device performance by maintaining better organization [4]. Detecting email spam is difficult because spammers are always changing their tactics, which makes it hard for filters to keep up. The characteristics of spam emails evolve over time, reducing the effectiveness of machine learning models. Additionally, spammers often craft emails specifically designed to evade detection systems [5] .

Machine learning is essential for spam detection because it can automatically learn and adapt to new spamming techniques, unlike traditional rule-based filters. It leverages algorithms to recognize patterns in large datasets, improving accuracy and efficiency in identifying spam emails. Additionally, machine learning models can handle the dynamic nature of spam, making them more robust against evolving threats [6].

1.1. Aim and Objective

The aim of this project is to develop an efficient email spam detection system using machine learning techniques that accurately classifies emails as spam or legitimate (ham) to enhance email security and minimize unwanted messages.

The objectives of this project are –

- To analyse the features of spam emails and identify patterns that differ them from legitimate emails.
- To implement machine learning models that can adapt to evolving spamming techniques.

- To train and test the dataset and model that can classify spam and non-spam emails.
- To develop a web app that detects and classifies emails as spam or non-spam using NLP.

1.2. Feasibility Study

The feasibility study starts with the following:

Economic Feasibility: The tools and techniques required for the email spam detection project are easily available and do not require any cost. Public datasets are free to use making the project economically feasible.

Technical Feasibility: To develop this project, we will use technical tools and frameworks like Python, Scikit-learn and NLTK which are open source and freely available and widely used for machine learning and natural language processing tasks.

Time Feasibility: We have made a realistic timeline for each phase of the project, including data collection, preprocessing, model training, testing and evaluation. The allocated times is sufficient for the project, making it time efficient.

Operational Feasibility: The proposed design will be kept simple and user-friendly with straightforward interface for user to input emails and receive classification result, that anyone with basic computer knowledge will be able to operate it, ensuring operational feasibility.

1.3. Literature Survey

Email Spam detection using NLP is mostly used in recent years as an approach to decrease the number of increasing malicious and unsolicited emails. Email Spam has caused threats to security, privacy as well as productivity. To tackle this problem, many researchers and developers have used Natural Language Processing, a type of AI that helps computers understand and comprehend human language. By using NLP, spam filter can classify the content of emails and identify patterns effectively. This literature survey explores how NLP and together with various techniques like supervised learning, unsupervised learning, semi-supervised learning or hybrid model and ensemble learning can be applied to email spam detection, highlighting the key methodologies, dataset used, the finding and research gaps. [7], [8]

1.3.1. Standalone Spam Detection Mechanisms

Kaddoura et al. (2020) [13] proposed a deep learning-based spam email detection mechanism focusing on English text emails, utilizing a Feed Forward Neural Network (FFNN) and the BERT transformer model. The system was trained and evaluated using the Enron dataset, achieving a high accuracy of 99.18%, demonstrating the effectiveness of combining traditional neural networks with modern transformer-based architectures. However, the approach has limitations, primarily its dependency on high GPU computational power, making real-time deployment impractical. Additionally, the model's generalizability is uncertain since it was only tested on the Enron dataset, highlighting the need for broader dataset validation and lightweight model alternatives. The Authors AbdulNabi and Yaseen in 2021[9] together presented an advanced spam email detection model using deep learning techniques, specifically BERT (Bidirectional Encoder Representation from Transformers). Their study aims to improve spam classification accuracy using word embedding and attention mechanisms. The approach is tested on two open-source datasets (Spambase and Kaggle Spam Filter dataset) obtaining an accuracy (98.67%) and F1-score (98.66%) outperforming conventional methods. However, BERT requires significant processing power and is only limited to a 300-word sequence, which may impact on longer emails. Likewise, Jáñez-Martino et al (2022) [5] proposed a study on A Review of spam email detection: Analysis of Spammer Strategies and the Dataset Shift Problem where he explored different machine learning model such as Naïve Bayes (NB), Support Vector Machine (SVM), Decision Tree (DT), Logistic Regression (LR), Neural Network (NN) and Deep learning (DL)[10]. This study was conducted using various publicly available datasets from 2010 and 2018. Their finding showed that the spam filter performed well on the older dataset, achieving 90% accuracy while it significantly dropped when tested on the recent dataset. This study also highlighted key challenges about the huge cost and the inability of traditional models to adapt to new spammer strategies and evolving email structures. Similarly, Zhang et al (2025) [11] proposed a combined feature selection approach for malicious email detection using a large and comprehensive dataset to address issues of outdated datasets and limited feature selection. To overcome these issues, they introduced the EPVME dataset, which contains 49,136 malicious email samples, simulating protocol vulnerability attacks (SPF, DMARC, S/MIME, OpenPGP). Their approach combines static feature extraction (headers, metadata) and NLP-based analysis (TF-IDF, Word2Vec) with machine learning models (Random Forest (RF), Decision Tree (DT), SVM, etc.). Random Forest with TF-IDF provided the highest accuracy (99.968%) and false positive rate of 0.099%. The dataset, however, does not have real-time updates and validation in real-world enterprise settings.

Table 1. Standalone Spam Detection Mechanisms

Title/Author Name	Methods Used	Dataset Used	Findings	Research Gaps
A Spam Email Detection Mechanism for English Language Text Emails Using Deep Learning Approach (Kaddoura et al, 2020) [12]	- FFNN (Feed Forward Neural Network) - BERT Transformer	Enron Dataset	Accuracy: 99.18%	It takes huge GPU power making real-time deployment challenging and it was only tested on Enron Dataset
Spam Email detection Using Deep Learning Techniques (AbdulNabi, Yaseen,2021)[9]	- BERT Transformer - BiLSTM - KNN	- Spambase - Kaggle	Accuracy: 98.67% F1-score: 98.66%	It only focuses on English emails and restricted to 300-word sequences
A Review of Spam Email Detection: Analysis of Spammer Strategies and the Dataset Shift Problem (Jáñez-Martino et al,2022) [5]	- Naïve Bayes (NB) - Support Vector Machine (SVM) - Decision Trees (DT) - Logistic Regression (LR) - Neural Networks (NN) - Deep Learning (DL)	- Ling-Spam - SpamAssassin - Enron-Spam - TREC07 - CSDM C - Bruce Guenter	Accuracy: 90% on older dataset	It requires high computational cost; the model also struggled with dataset shift and evolving spammer strategies
A Combined Feature Selection Approach for Malicious Email Detection (Zhang et al, 2025) [11]	Static & NLP-based feature extraction. Machine learning models (Random Forest, Decision Tree, Support Vector Machine, etc.)	EPVME dataset (49,136 malicious emails)	Random Forest with TF-IDF achieved 99.968% accuracy, 0.099% FPR.	Dataset lacks real time updates of enterprise data. Risk of potential data imbalance.

1.3.2. Hybrid Model

Krishnamoorthy et al (2024) [13] proposed a hybrid deep neural network (DNN-BiLSTM) for email classification to enhance spam detection and tackle security issues. The proposed method involves feature extraction, preprocessing, and classification with BiLSTM, CNN, LSTM, and Random Forest. The DNN-BiLSTM model accomplished the highest accuracy of 98.69% on the combined Enron dataset. However, it is missing real-world testing on varied email datasets, limiting its generalizability. To improve this model, we can employ ensemble classification techniques that could further optimize security and performance. Similarly, Iqbal et al (2025) [14] proposed a hybrid spam detection model (Hybrid-BoostSVM) that integrates spammer behavior features and linguistic features to improve accuracy in detecting fake reviews. The study highlights the challenges of review sampling, where fraudulent reviews are posted to manipulate online ratings. Unlike existing models that focus solely on textual or behavioral features, this model fuses both feature types using (GB and SVM) for classification. The model was tested on a large-scale Amazon review dataset, achieving 94.6% accuracy, demonstrating its effectiveness over traditional approaches.

Table 2. Hybrid Model

Title/Author Name	Methods Used	Dataset Used	Findings	Research Gaps
A Novel and Secured Email Classification and Emotion Detection Using Hybrid Deep Neural Network (Krishnamoorthy et al 2024) [13]	DNN-BiLSTM	Enron Dataset (6 subsets, combined dataset: 33,727 emails)	DNN-BiLSTM achieved 98.69% accuracy)	Limited real-world testing on diverse dataset.
Email Spam Detection Using Hybrid Model on Fusion of Spammer Behavior and Linguistic Features (Iqbal et al 2025) [14]	Hybrid-BoostSVM (Gradient Boosting + SVM)	Amazon Product Review Dataset	Accuracy: 94.6%	Only tested on Amazon reviews, lacking multilingual evaluation.

1.3.3. Ensemble Model

Adnan et al (2023) [15] explored the use of ensemble learning using stacking approach which is a technique that combines predictions from multiple machine learning models to improve classification accuracy. The base classifier used in this study included Logistic Regression (LR), Decision tree (DT), Naïve Bayes (NB), K-

Nearest Neighbors (KNN) and AdaBoost. The research was conducted using a combination of two openly available email datasets i.e. SpamAssassin and Enron-Spam. The findings show that the stacking ensemble method got an accuracy of 98.8%, recall of 98.8%, and an F1-score of 98.9%. However, this model suffers from the potential need for more diverse datasets to further generalize the model.

Table 3. Ensemble Model

Title/Author Name	Methods Used	Dataset Used	Findings	Research Gaps
Improving Spam Email Classification Accuracy Using Ensemble Techniques: A Stacking Approach (Adnan et al 2023) [15]	- Logistic Regression - Decision Tree - Naïve Bayes - K-Nearest Neighbor - AdaBoost	- SpamAssassin - Enron-Spam datasets	Stacking method achieved 98.8% accuracy, 98.8% recall, and 98.9% F1-score	The model requires more diverse datasets for better generalization.

1.4. Problem Definition

The rise of email communication has led to a surge in spam messages, which are unsolicited and potentially harmful. These emails clutter inboxes while wasting time and enables security threats such as phishing and malware attacks. Traditional rule-based filters struggle to keep up with evolving spam tactics, making them less effective. As a result, machine learning (ML) has become a more effective solution for spam detection, as ML models can analyse patterns in email data, adapt to new threats, and deliver more accurate classification between spam and legitimate emails. Implementing an ML-based system can enhance email security, reduce user frustration, and optimize resource utilization.

1.5. Analysis of the Problem

Spam messages are one of the biggest problems with electronic communication, typically containing unsolicited advertisements, phishing scams, and malicious attachments, which represent security threats such as data interception and malware infection. The most significant problem is that spammers continually change their methods, and standard rule-based filters are powerless against them. Furthermore, anti-spam systems must reconcile accuracy, with low false positives (legitimate emails marked as spam) and false negatives (missed spam emails). Scalability is also a point of concern because email systems deal with large amounts of data and need to use efficient

and adaptable models. Pattern identification through word frequency, sender information, and structure of emails is achieved with feature extraction. Although rule-based filtering is straightforward, it can easily be circumvented, whereas machine learning filters such as Naïve Bayes and SVM are more accurate but need updating on an ongoing basis. More advanced solutions that combine rule-based and machine learning algorithms deliver superior detection but require high computational power. To effectively counter changing spam tactics, the system needs to learn dynamically from new information and utilize machine learning with real-time updates for superior accuracy and efficiency.

1.6. Purposed Solution Strategy

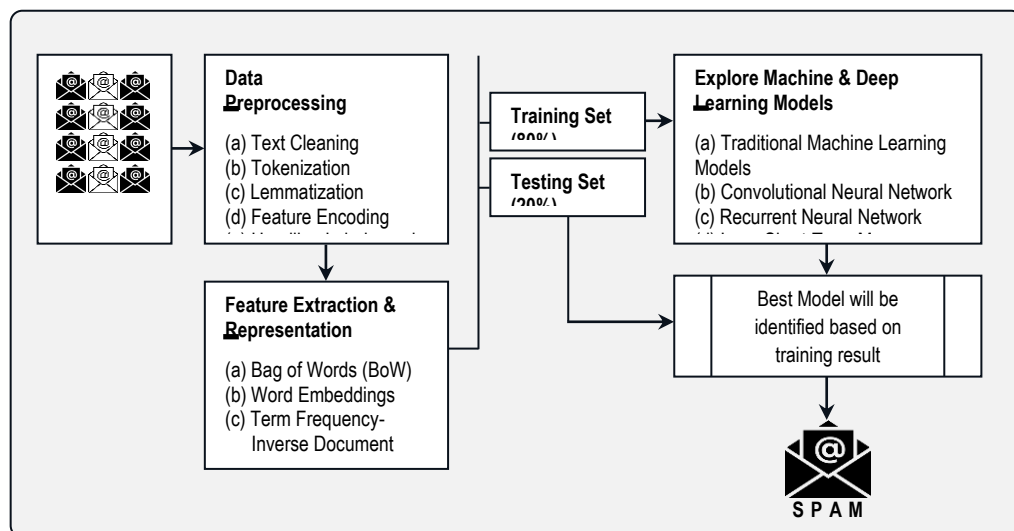


Figure 1. Purposed Solution Strategy

2. PROJECT PLANNING

2.1. Hardware and Software Requirements

The hardware and software requirements to execute the project are –

(a) Hardware:

Processor: Intel core i3 7th Gen

Memory: 4 GB DDR4 RAM

Storage: 256 GB+

(b) Software:

Windows 7 and above

Python 3.11.3

Pycharm

2.2. Team Structure

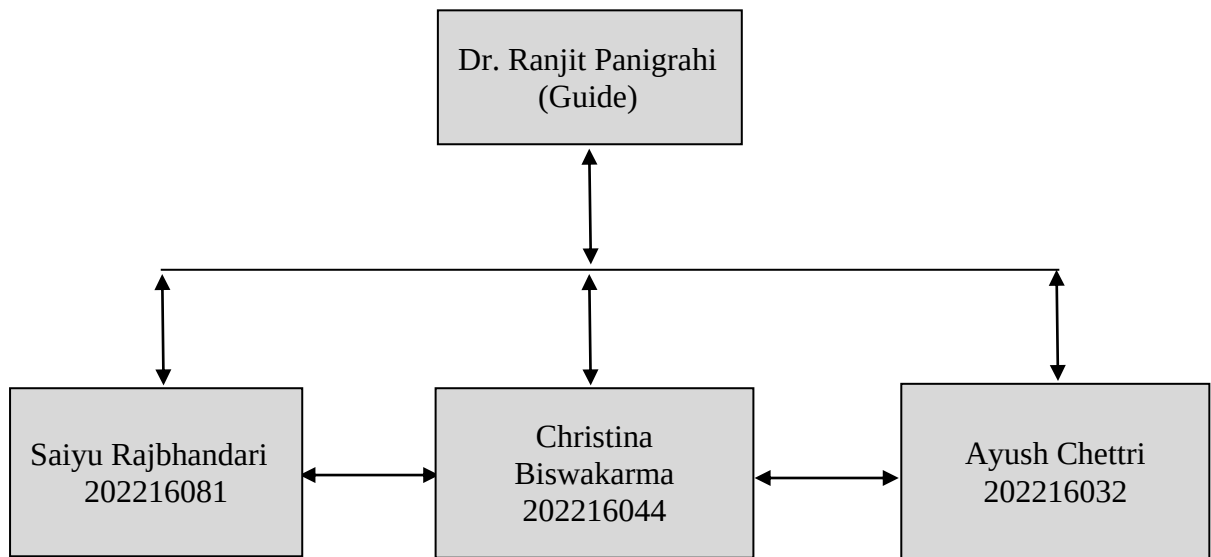


Figure 2. Team Structure

2.3. Gantt Chart

Activities	14-01-2025 07-01-2025	08-02-2025 06-03-2025	07-03-2025 06-03-2025	07-04-2025 23-04-2025
Literature Survey	Target	Achieved	Achieved	Achieved
Problem Formulation	Target			
Data Pre-Processing		Target		
Feature Selection			Target	
Classification & Detection			Target	
Result & Analysis				Target
Documentation	Target	Target	Target	Target

Target

Achieved

Figure 3. The Project Schedule

3. IMPLEMENTATION DETAILS

The implementation of email spam detection is as follows:

3.1. Data Collection

This project used an email dataset sourced from GitHub repository which contains csv file of emails with both spam and ham. The email.csv file contained 5726 rows and 2 columns with columns text and spam. The text included the subject name and the email information whereas the spam showed values of ham and spam denoted by 0 and 1 respectively.

3.2. Data Preprocessing

Data Pre-processing is necessary to handle datasets so that we get accurate results. Data pre-processing helps make data more refined by eliminating all missing values, duplicate values and inconsistent values. The data preprocessing was done in following steps:

3.2.1. Importing Libraries

Commonly used libraries in machine learning and data science projects such as NumPy, Pandas, Matplotlib, and Scikit-learn were used which provide support for numerical computations, data manipulation, visualization, and model implementation. In text processing, string and nltk (Natural Language Toolkit) are used to work with text data, including downloading stopwords, which are common words that are usually removed in text analysis. Additionally, machine learning and data processing libraries are included, such as TfidfVectorizer for converting text into numerical form, SMOTE for handling imbalanced data, and train_test_split for dividing data into training and testing sets.

```
## Data Handling
import pandas as pd
import numpy as np

# Data Visualization
import seaborn as sns
import matplotlib.pyplot as plt

#Text Processing
import string
import nltk
from nltk.corpus import stopwords
nltk.download('stopwords')

from sklearn.feature_extraction.text import TfidfVectorizer
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split
```

Figure 4. Importing Libraries

3.2.2. Data loading

The next step is loading a dataset named `emails.csv` using Pandas. The dataset has 5,726 rows and 2 columns, as seen from the `shape` function. A sample of five random records is displayed using `sample(5)`, showing email subjects and a spam label (0 or 1), where 0 indicates non-spam emails.

```
# Load the dataset
email11 = pd.read_csv("emails.csv")
email11.shape
```

[5]:

```
(5726, 2)
```

[7]:

```
# Show dataset sample (5 random records)
email11.sample(5)
```

[7]:

	text	spam
2896	Subject: thursday instead friday john sherrif...	0
1979	Subject: steve ' s trip to houston dale , ric...	0
305	Subject: are you ready to get it ? hello ! v...	1
2510	Subject: hea sporting clays tournament - augus...	0
1094	Subject: great news . youve been pre - approve...	1

Figure 5. Loading Dataset

3.2.3. Data Visualization

This step uses a bar chart which visualizes the distribution of spam vs non-spam emails in the dataset. The green bar represents non-spam emails (0), while the red bar represents spam emails (1). The dataset has significantly more non-spam emails than spam. A warning appears is shown around a Seaborn parameter (`palette` without `hue`) to indicate that it may need updating in future versions.



Figure 6. Data Visualization

3.2.4. Data Cleaning

The data cleaning steps performed on the dataset firstly checks for missing values, confirming that both the "text" and "spam" columns have no null values. Next, it checks for duplicate entries, finding 33 duplicate rows. Finally, it removes these duplicates using `drop_duplicates(inplace=True)`, ensuring cleaner data for further processing.

```
# Check for missing data for each column
email1.isnull().sum()

text    0
spam    0
dtype: int64

# Check for duplicates
email1.duplicated().sum()

33

# Remove the duplicates
email1.drop_duplicates(inplace=True)
```

Figure 7. Data Cleaning

3.2.5. Tokenization

The image below shows the text preprocessing and feature extraction steps for an email dataset. The process function first removes punctuation from the text by filtering out characters that belong to `string.punctuation`. Then, it removes stopwords (common words like "the", "is", "and") using the `stopwords.words('english')` list. After this cleaning process, the text is split into

11

individual words (tokens). The cleaned and tokenized text is then transformed into a numerical format using the TF-IDF(Term Frequency-Inverse Document Frequency) technique, implemented through the TfidfVectorizer(). This method assigns weights to words based on their frequency within a single email versus across the entire dataset, emphasizing more relevant terms. The fit_transform() function processes and transforms the "text" column into a matrix of numerical features for input into machine learning models.

```
# Cleaning data for punctuation and stopwords and then tokenizing it into words(tokens)
def process(text):
    nopunc = [char for char in text if char not in string.punctuation]
    nopunc = ''.join(nopunc)
    clean = [word for word in nopunc.split() if word.lower() not in stopwords.words('english')]
    return clean

# Fit the TfidfVectorizer to data
vectorizer = TfidfVectorizer()
message = vectorizer.fit_transform(email['text'])
```

Figure 8. Tokenization

3.2.6. Data Balancing

Due to imbalance in dataset, SMOTE (Synthetic Minority Over-sampling Technique) can be applied to the training data to generate synthetic spam samples to ensure an equal number of spam and non-spam emails. The process include selecting features, splitting the dataset, and handling class imbalance using SMOTE for an email spam detection model. The email text is transformed into numerical features (X), while the spam labels (y) indicate whether an email is spam (1) or not (0). The dataset is then split into training and testing sets, with 20% of the data reserved for testing. This approach ensures that the model is trained on balanced data, which helps improve its ability to detect spam effectively and prevents it from being biased.

```
# Features (email text) and target (spam)
X = message
y = email['spam']
# Splitting the dataset before oversampling
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Applying SMOTE to training data
smote = SMOTE()
X_train_resampled, y_train_resampled = smote.fit_resample(X_train.toarray(), y_train)
# Checking class distribution
print(pd.Series(y_train_resampled).value_counts())

spam
0    3482
1    3482
Name: count, dtype: int64
```

Figure 9. Data Balancing

3.3. Holdout Cross Validation

4 To evaluate model performance, the dataset was divided into two parts: 80% training data and 20% testing. This method ensure that the model is trained on most of the data while keeping a separate portion for evaluation, preventing data leakage and ensuring unbiased performance assessment. The training portion is used to train the machine learning model that allows it to learn underlying patterns and association within the extracted features, while the testing set is used to evaluate its generalization ability on unseen data. This strategy is essential for calculating the performance metrics, such as accuracy, precision, recall, and F1-score, ensuring that it can accurately classify between spam and legitimate emails.

```
# Holdout CrossValidation
X_train, X_val, y_train, y_val = train_test_split(X_train_combined, y_train_comb

print("X_train shape:", X_train.shape)
print("X_val shape:", X_val.shape)
print("y_train shape:", y_train.shape)
print("y_val shape:", y_val.shape)

print("\nX_train example:\n", X_train[:5])
print("\nX_val example:\n", X_val[:5])
```

Figure 10. Holdout Cross-Validation

3.4. Explored Various Classification Models

13 **Traditional Models:** In traditional model, various models were used. Naïve Bayes (NB) which predicts class labels by calculating probabilities (using Bayes' theorem) and assuming features are independent. In addition, Support Vector Machine (SVM) finds the best boundary (hyperplane) that separates different classes with the greatest possible margin. Decision Trees (DT) splits data into branches using "if-else" rules to make decisions like a flowchart. Similarly, Random Forest (RF), an ensemble technique, builds multiple decision trees and averages their predictions, and reduces overfitting. Likewise, Gradient Boosting (GB) also uses an ensemble approach but differs by sequentially building models; each new tree is trained to correct the errors of the previous one. Gradient Boosting stands out with the highest accuracy (0.974753), precision (0.953052), recall (0.939815), F1-score (0.946387), and a near-perfect AUC (0.997702), making it the best-performing model overall. Random Forest also performs well across most metrics but slightly trails behind Gradient Boosting. Naive Bayes has high precision but lower recall and F1-score, suggest difficult in dealing with imbalanced datasets. SVM and Decision Tree show strong results, but Gradient Boosting clearly outshined in overall effectiveness.

```
# Define models
models = {
    "Naive Bayes": MultinomialNB(),
    "SVM": SVC(probability=True),
    "Decision Tree": DecisionTreeClassifier(),
    "Random Forest": RandomForestClassifier(),
    "Gradient Boosting": GradientBoostingClassifier()
}

# Store results
results = []

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_val)
    y_prob = model.predict_proba(X_val)[:, 1] if hasattr(model, "predict_proba")

    accuracy = accuracy_score(y_val, y_pred)
    precision = precision_score(y_val, y_pred)
    recall = recall_score(y_val, y_pred)
    f1 = f1_score(y_val, y_pred)
    auc = roc_auc_score(y_val, y_prob) if y_prob is not None else "N/A"

    results.append([name, accuracy, precision, recall, f1, auc])
print("Traditional ML models evaluated successfully!")
```

	Model	Accuracy	Precision	Recall	F1-Score	AUC
0	Naive Bayes	0.920966	0.980000	0.680556	0.803279	0.996270
1	SVM	0.949506	0.837302	0.976852	0.901709	0.993265
2	Decision Tree	0.960483	0.920561	0.912037	0.916279	0.943788
3	Random Forest	0.960483	0.994505	0.837963	0.909548	0.997808
4	Gradient Boosting	0.974753	0.953052	0.939815	0.946387	0.997702

Figure 11. Traditional Classification Model

Deep Learning Model: Unlike traditional model, deep learning model worked better at getting better results. Starting with the Convolutional Neural Network (CNN), which captured spatial patterns in text data by treating word embeddings as feature maps. Following this, Recurrent Neural Networks (RNN) analyzed sequential dependencies in emails, enabling the model to capture contextual relationships between words. Additionally, Long Short-Term Memory (LSTM) networks were used handle long-range reliances in text, making them suitable for processing email content where spam indicators may be distributed throughout the message. Lastly, the Gated Recurrent Unit (GRU), a variation of LSTM with a simplified architecture, was utilized to improve efficiency while retaining the ability to capture sequential dependencies. By comparing the performance of these models, CNN performs the best overall, achieving the highest accuracy (0.9772), F1-score (0.9555), and AUC (0.9979), making it the most reliable model for balanced performance in spam detection. RNN shows the highest precision (0.9781), indicating it's best at minimizing false positives, but its recall is comparatively lower (0.9054), which means it misses more actual spam instances. LSTM and GRU show balanced performance, with LSTM slightly

better in recall and F1-score, whereas GRU has the lowest AUC and F1-score of the four.

```
# Training and Evaluation
results = []
model_types = ["CNN", "RNN", "LSTM", "GRU"]

for model_type in model_types:
    print(f"Training {model_type} model...")
    model = create_model(model_type)

    model.fit(X_train, y_train, epochs=5, batch_size=32, verbose=0,
              validation_data=(X_test, y_test), # Use test set for validation
              callbacks=[EarlyStopping(monitor='val_loss', patience=1, restore_b

    y_pred = (model.predict(X_test, verbose=0) > 0.5).astype("int32")
    y_prob = model.predict(X_test, verbose=0).flatten()

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, zero_division=0)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    auc = roc_auc_score(y_test, y_prob)

    results.append([model_type, accuracy, precision, recall, f1, auc])
```

	Model	Accuracy	Precision	Recall	F1-Score	AUC
0	CNN	0.977173	0.968750	0.942568	0.955479	0.997936
1	RNN	0.970149	0.978102	0.905405	0.940351	0.996289
2	LSTM	0.972783	0.940199	0.956081	0.948074	0.996201
3	GRU	0.970149	0.948630	0.935811	0.942177	0.994433

Figure 12. Deep Learning Mode

3.5. K-fold Cross Validation on CNN

K-Fold Cross Validation is a technique used to evaluate the performance of a machine learning model by splitting a dataset into K equal parts i.e. folds. In the present project, we have applied 10-Fold Cross Validation to determine the reliability and generalizability of the CNN model. The dataset was divided into 10 folds and in every run, the model was trained on 9 of them and validated on the rest 1. This process was repeated ten times to allow each fold to act as a validation set one time. Loss measures the error of the model's prediction with lower values being better and accuracy is the proportion of the correct predictions made. The response indicated that accuracy was always above 97.8 %, and several folds registered as high as 99.3%. Moreover, the loss values were kept below 0.06 in all folds, showing that the CNN model was able to have low values and high stability in throughout the validation process. The following table shows the loss and accuracy obtained in each fold:

Fold 1 - Loss: 0.0370, Accuracy: 0.9878
 Fold 2 - Loss: 0.0596, Accuracy: 0.9791
 Fold 3 - Loss: 0.0439, Accuracy: 0.9825
 Fold 4 - Loss: 0.0544, Accuracy: 0.9825
 Fold 5 - Loss: 0.0255, Accuracy: 0.9930
 Fold 6 - Loss: 0.0246, Accuracy: 0.9930
 Fold 7 - Loss: 0.0478, Accuracy: 0.9878
 Fold 8 - Loss: 0.0293, Accuracy: 0.9930
 Fold 9 - Loss: 0.0395, Accuracy: 0.9860
 Fold 10 - Loss: 0.0335, Accuracy: 0.9895

Figure 13. K-fold Cross Validation Result (Loss and Accuracy)

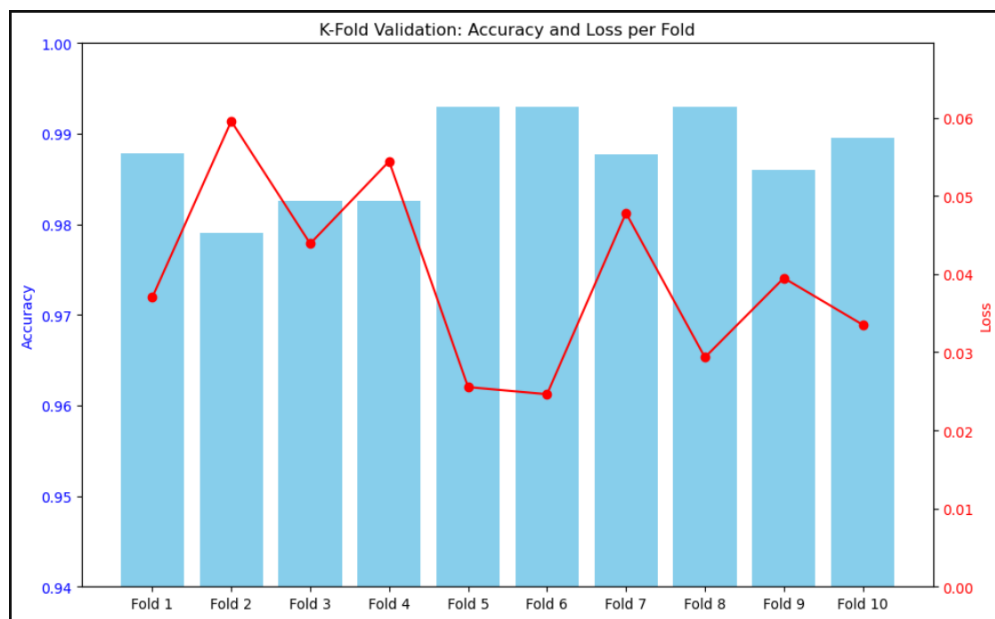


Figure 14. K-fold Cross Validation Graph (Loss and Accuracy)

3.6. Saving model and tokenizer

Similarly, the next step involves the process of saving and reusing a trained CNN model along with its tokenizer, which is essential for deploying the model in a Streamlit web application built using PyCharm. The tokenizer is responsible for converting input text into numerical sequences which later is serialized and saved using the pickle module with the highest protocol to ensure compatibility and efficiency. Similarly, the trained CNN model is saved in the .keras format using Keras' built-in `.save()` function. For later use, it can be reloaded using `load_model()` and even renamed or copied using Python's `shutil` module, which is helpful for organizing model versions. In the Streamlit app, user input is first preprocessed using the loaded tokenizer, then passed into the CNN model to predict outcomes—such as classifying whether an email is spam or not. This workflow makes the model accessible through a clean, interactive web interface,

allowing real-time predictions without retraining or redefining preprocessing steps.

```
cnn_model.save('cnn_model.keras')

model = load_model('cnn_model.keras')
import shutil
shutil.copy('cnn_model.keras', 'cnn_model.mds')
```

Figure 15. Saving Model

```
import pickle
with open('tokenizer.pickle', 'wb') as handle:
    pickle.dump(tokenizer, handle, protocol=pickle.HIGHEST_PROTOCOL)
```

Figure 16. Saving Tokenizer

3.7. Developing Web App for Email Classification Using Streamlit

To make a website, the trained Convolutional Neural Network (CNN) model and tokenizer are integrated into a Streamlit web application for spam detection. First, the necessary modules like streamlit, pickle, and keras are imported. The saved tokenizer (in .pickle format) and CNN model (in .keras format) are then loaded into the application. When the user enters email text into the web interface and clicks the "Check" button, the text is first tokenized and padded to keep up the model's input shape. The CNN model then alerts whether the email is spam or not. If the prediction probability is greater than 0.5, the email is classified as "Spam"; otherwise, it's labelled as "Not Spam". This real-time interface offers a simple and interactive way for users to test emails, making it a practical implementation of NLP and deep learning using Streamlit in PyCharm.

```
import streamlit as st
import pickle
from keras.models import load_model
from keras.preprocessing.sequence import pad_sequences

# Load tokenizer and model
tokenizer = pickle.load(open("tokenizer2.pickle", "rb"))
model = load_model("cnn_model.keras")
```

Figure 17. Importing modules and loading model

```
st.markdown('<p class="title"> 📧 Email Spam Detector </p>', unsafe_allow_html=True)

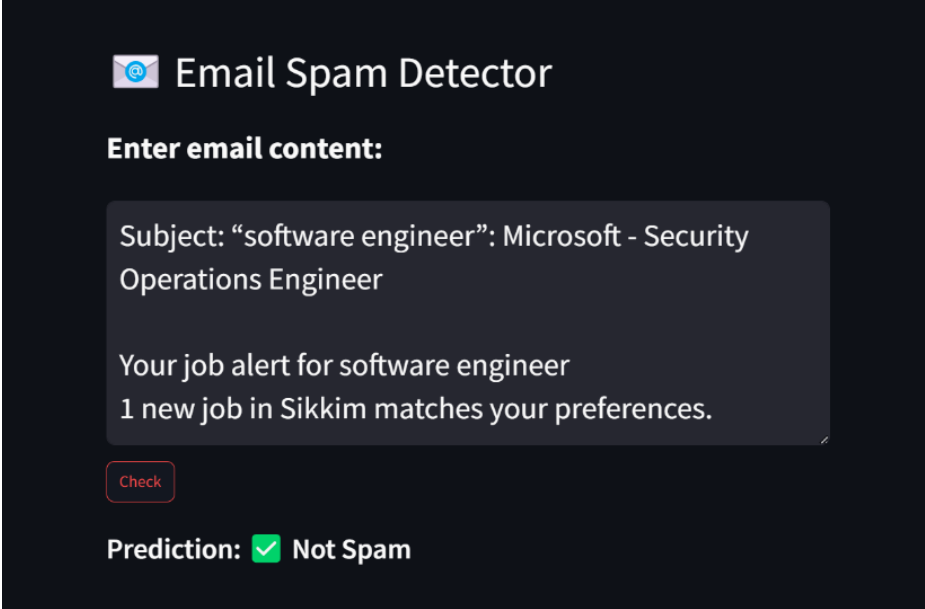
st.write('<p class="custom-label">Enter email content:</p>', unsafe_allow_html=True)
email_text = st.text_area("", height=200)

if st.button("Check"):
    if email_text.strip() == "":
        st.warning("Please enter some text.")
    else:
        sequence = tokenizer.texts_to_sequences([email_text])
        padded = pad_sequences(sequence, maxlen=100)
        prediction = model.predict(padded)[0][0]
        label = "🚫 Spam" if prediction > 0.5 else "✅ Not Spam"
        st.subheader(f"Prediction: {label}")
```

Figure 18. Adding Input box and button

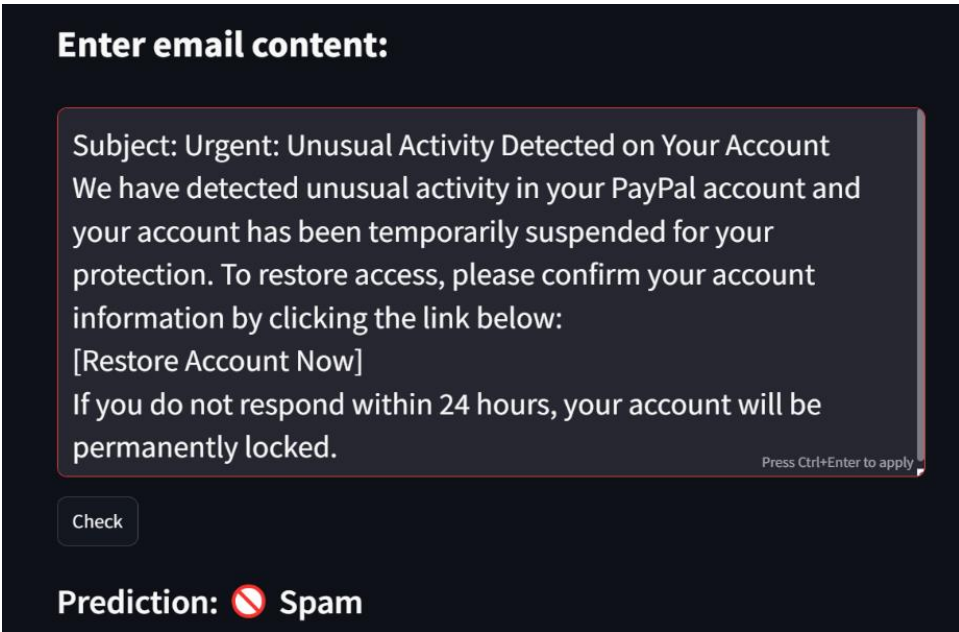
4. RESULT AND DISCUSSION

4.1. Spam Classification Result for Sample Email Input



The screenshot shows a web interface titled "Email Spam Detector" with an email icon. Below the title is a label "Enter email content:". A text area contains the following text: "Subject: 'software engineer': Microsoft - Security Operations Engineer", "Your job alert for software engineer", and "1 new job in Sikkim matches your preferences.". Below the text area is a red "Check" button. At the bottom, the prediction is displayed as "Prediction: ✓ Not Spam".

Figure 19. Prediction Result for User Input Email: "**Not Spam**"



The screenshot shows the same "Email Spam Detector" interface. The text area contains a phishing email: "Subject: Urgent: Unusual Activity Detected on Your Account", "We have detected unusual activity in your PayPal account and your account has been temporarily suspended for your protection. To restore access, please confirm your account information by clicking the link below:", "[Restore Account Now]", and "If you do not respond within 24 hours, your account will be permanently locked.". Below the text area is a red "Check" button. At the bottom, the prediction is displayed as "Prediction: ✗ Spam".

Figure 20. Prediction Result for User Input Email: "**Spam**"

Description: In figure 19 we can see a sample email about a job alert, where the system correctly predicts the email as **Not Spam**. Also, in figure 20 we can see a suspicious email claiming unusual PayPal activity, and the system successfully classifies it as **Spam**. This demonstrates that the spam detector can differentiate between legitimate and phishing-type emails effectively.

5. SUMMARY AND CONCLUSIONS

5.1. Difficulty Encountered During Project

Since the project was research based, a lot of effort went into studying research papers, and literature survey to clearly define the project's direction. Another major difficulty was finding a suitable and high-quality dataset, which involved searching through multiple open-source repositories and databases. Additionally, the project required learning new technical terms, tools, and software, which added to the complexity. Implementing new technologies while progressing with the project was challenging and needed extra time and effort.

5.2. Future Scope of the Project

In the future, this project can be improved by using advanced model like BERT, which understand the meaning of words better and can catch more clever phishing attacks. We can also ensemble different deep learning models (RNN, LSTM, and GRU) in the project model i.e. CNN and add extra information like the sender's reputation or links inside the email, to make the system even more accurate. Building a system that keeps learning over time by retraining itself with new emails will help the model stay updated against new types of spam and also adding a feature where users can report wrong detections (like marking spam that was missed or emails wrongly flagged) will help improve the model by correcting the training data regularly.

5.3. Limitations of the Project

Even though the project was completed successfully, there were certain limitations that need to be acknowledged. The most significant issue was related to the dataset used for training the model. Since the dataset was quite small and didn't cover a wide enough range of scenarios, the model ended up overfitting. It performed very well on the training and testing data provided but struggled when it came to unseen or new data. This shows that the model had learned the patterns specific to the given dataset rather than truly understanding the overall problem.

5.4. Conclusion

This project successfully demonstrated how machine learning techniques can be used for accurate detection and classification spam emails. By collecting and cleaning a labeled dataset of spam and non-spam emails, converting the text into numerical features using TF-IDF, and training various traditional classification model—including Naïve Bayes, SVM, Decision Tree, Random Forest and Gradient Boosting, and together with deep learning models such as CNN, RNN, LSTM, and GRU, we

were able to compare their performances using key metrics like accuracy, precision, recall, F1-score, and AUC. Among these models, CNN showed the most promising results. We also built a user-friendly web application using streamlit in Pycharm, allowing users to check if an email is spam or not. Overall, this project helped us understand the practical aspects of natural language processing, deep learning, and web deployment in solving real-world problems.

REFERENCES

- [1] “Why is email so important to modern communication?” Accessed: Jan. 27, 2025. [Online]. Available: <https://adguard.com/en/blog/why-is-email-so-important-to-modern-communication.html>
- [2] “What is email spam and how to fight it? | Definition from TechTarget.” Accessed: Jan. 27, 2025. [Online]. Available: <https://www.techtarget.com/searchsecurity/definition/spam>
- [3] “2023 Data Breach Investigations Report: frequency and cost of social engineering attacks skyrocket | News Release | Verizon.” Accessed: Jan. 27, 2025. [Online]. Available: <https://www.verizon.com/about/news/2023-data-breach-investigations-report>
- [4] “Why Should You Clear Out Your Emails?” Accessed: Jan. 27, 2025. [Online]. Available: <https://mailstrom.co/articles/why-should-you-clear-out-your-emails/>
- [5] F. Jáñez-Martino, R. Alaiz-Rodríguez, V. González-Castro, E. Fidalgo, and E. Alegre, “A review of spam email detection: analysis of spammer strategies and the dataset shift problem,” *Artif Intell Rev*, vol. 56, no. 2, pp. 1145–1173, Feb. 2023, doi: 10.1007/S10462-022-10195-4.
- [6] “Email Spam Detection Using Machine Learning Algorithms.” Accessed: Jan. 27, 2025. [Online]. Available: <https://www.jetir.org/view?paper=JETIR2405G48>
- [7] “Enhancing Software Solutions with Natural Language Processing | MoldStud.” Accessed: May 10, 2025. [Online]. Available: <https://moldstud.com/articles/p-leveraging-natural-language-processing-in-software-solutions>
- [8] “U.S. Patent for Cell control system, manufacturing system, and control method which control manufacturing cell including plurality of manufacturing machines Patent (Patent # 9,971,329 issued May 15, 2018) - Justia Patents Search.” Accessed: May 10, 2025. [Online]. Available: <https://patents.justia.com/patent/9971329>
- [9] I. AbdulNabi and Q. Yaseen, “Spam Email Detection Using Deep Learning Techniques,” *Procedia Comput Sci*, vol. 184, pp. 853–858, Jan. 2021, doi: 10.1016/J.PROCS.2021.03.107.
- [10] M. Alkasassbeh, “An empirical evaluation for the intrusion detection features based on machine learning and feature selection methods,” *J Theor Appl Inf Technol*, vol. 95, no. 22, pp. 5962–5976, Nov. 2017.

- [11] H. Zhang, Y. Shi, M. Liu, L. Chen, S. Wu, and Z. Xue, “A combined feature selection approach for malicious email detection based on a comprehensive email dataset,” *Cybersecurity*, vol. 8, no. 1, pp. 1–22, Dec. 2025, doi: 10.1186/S42400-024-00309-6/TABLES/16.
- [12] “Sci-Hub | A Spam Email Detection Mechanism for English Language Text Emails Using Deep Learning Approach. 2020 IEEE 29th International Conference on Enabling Technologies: Infrastructure for Collaborative Enterprises (WETICE) | 10.1109/WETICE49692.2020.00045,” 2020, Accessed: Mar. 06, 2025. [Online]. Available: <https://sci-hub.st/10.1109/WETICE49692.2020.00045>
- [13] P. Krishnamoorthy, M. Sathiyarayanan, and H. P. Proença, “A novel and secured email classification and emotion detection using hybrid deep neural network,” *International Journal of Cognitive Computing in Engineering*, vol. 5, pp. 44–57, Jan. 2024, doi: 10.1016/J.IJCCE.2024.01.002.
- [14] A. Iqbal, M. Younas, S. Iftikhar, F. Fatima, and R. Saleem, “Spam detection using hybrid model on fusion of spammer behavior and linguistics features,” *Egyptian Informatics Journal*, vol. 29, p. 100605, Mar. 2025, doi: 10.1016/J.EIJ.2024.100605.
- [15] M. Adnan, M. O. Imam, M. F. Javed, and I. Murtza, “Improving spam email classification accuracy using ensemble techniques: a stacking approach,” *Int J Inf Secur*, vol. 23, no. 1, pp. 505–517, Feb. 2024, doi: 10.1007/S10207-023-00756-1/FIGURES/7.